AI Vibe Coded Design Pattern Guide

Overview

A practical design pattern for business application developers building their first production app. This guide bridges the [Enterprise AI Application Architecture Standards] with real-world implementation, calling out where the standards apply, where they flex, and what matters most for a solid foundation.

Audience: Business vibe coders stepping into their first application build with Claude Code or similar AI-assisted development tools.

Goal: Cover 80% of general business applications with patterns that are secure, maintainable, and production-ready -- without over-engineering.

How to Read This Guide

Each section follows this format:

- **Standard Says** -- What the Enterprise Architecture Standards prescribe
- **In Practice** -- What makes sense for most business apps
- **When to Deviate** -- Valid reasons to take a different path
- **Getting Started** -- The minimum viable implementation

---

1. Authentication: OIDC / Azure AD SSO

Standard Says

All applications MUST use Azure AD (Entra ID) for authentication via OIDC. No custom password authentication. Use NextAuth.js with Azure AD provider.

In Practice

The principle is correct and non-negotiable: **delegate authentication to your identity provider.** Your app should never store, manage, or validate passwords.

The specific library (NextAuth.js vs. custom OIDC) is flexible based on your stack:

Stack	Recommended Library	Why
Next.js (full-stack)	NextAuth.js	50 lines, built-in session management
NestJS / Express	`openid-client` v5	Native Node.js OIDC client
FastAPI / Python	`authlib`	Standard Python OIDC library
.NET	Microsoft.Identity.Web	Microsoft's own library

All of these implement the same OIDC Authorization Code flow with PKCE. The protocol is the standard -- the library is the tool.

When to Deviate

- **NextAuth.js** is ideal when your frontend IS the backend (Next.js API routes). If your backend is a separate service (NestJS, FastAPI), NextAuth.js doesn't apply -- use the library native to your backend framework.
- **Mock OIDC for local dev** is recommended. Don't require a real Azure AD tenant for development. Build or use a mock IdP that simulates the same flow.

Getting Started

Minimum viable OIDC implementation:

1. Register an Azure AD app (or use mock-oidc for dev)
2. Implement three endpoints: `/auth/login`, `/auth/callback`, `/auth/me`
3. Store session with `created_at` and `last_activity` timestamps
4. Support at least one production provider (Entra ID) and one dev provider (mock-oidc)

```
Browser -> /auth/login -> Redirect to IdP
IdP -> /auth/callback -> Exchange code for tokens -> Create session
Browser -> /auth/me -> Return user info from session
```

Key Principle: Azure AD handles AUTHENTICATION (who are you?), your application handles AUTHORIZATION (what can you do?). These are separate concerns -- never mix them.

---

2. Authorization: Permission-Based RBAC

Standard Says

Code checks PERMISSIONS, not roles. Roles are collections of permissions. Use a centralized PERMISSIONS configuration. Anti-pattern: `if (user.role === 'admin')`.

In Practice

This is the highest-impact pattern in the standards. Permission-based access control is **strongly recommended** even for simple apps because:

- Adding a new role takes 5 minutes (add to config) vs. hours (find every role check in code)
- Security audits can review one file instead of grep-ing the entire codebase
- You can answer "who can delete users?" by reading a config, not tracing code paths

Minimum viable implementation -- one file:

```python
# Python (FastAPI)
# lib/permissions.py

PERMISSIONS = {
    "can_access_admin":   ["super_admin", "admin"],
    "can_manage_users":   ["super_admin", "admin"],
    "can_manage_settings":["super_admin"],
    "can_manage_api_keys":["super_admin", "admin"],
    "can_view_reports":   ["super_admin", "admin", "user"],
    "can_use_app":        ["super_admin", "admin", "user"],
}

def has_permission(user_role: str, permission: str) -> bool:
    return user_role in PERMISSIONS.get(permission, [])

def require_permission(user_role: str, permission: str):
    if not has_permission(user_role, permission):
```

```
        raise HTTPException(status_code=403, detail=f"Missing permission: {permission}")
```

```typescript
// TypeScript (NestJS / Next.js)
// lib/permissions.ts

export const PERMISSIONS = {
  canAccessAdmin:    ["SuperAdmin", "Admin"],
  canManageUsers:    ["SuperAdmin", "Admin"],
  canManageSettings: ["SuperAdmin"],
  canManageApiKeys:  ["SuperAdmin", "Admin"],
  canViewReports:    ["SuperAdmin", "Admin", "User"],
  canUseApp:         ["SuperAdmin", "Admin", "User"],
} as const;

export function hasPermission(
  userRole: string,
  permission: keyof typeof PERMISSIONS
): boolean {
  return PERMISSIONS[permission].includes(userRole as any);
}
```

Usage in routes:

```python
# Instead of: @Roles(AppRole.Admin)
# Do:
@router.delete("/api/admin/users/{id}")
async def delete_user(id: str, user = Depends(get_current_user)):
    require_permission(user.role, "can_manage_users")
    ...
```

When to Deviate

- **Prototype / hackathon (< 1 week lifespan):** Simple role checks are fine. But if it might go to production, start with permissions.
- **Single-role apps:** If your app only has "authenticated" and "unauthenticated" users, role checks are sufficient.

Getting Started

1. Create a single `permissions` file (see above)
2. Define your roles: start with 3 (`super_admin` , `admin` , `user`)
3. Define permissions based on what users DO, not who they ARE
4. Replace all `if role === 'admin'` checks with `has_permission(role, 'permission_name')`
5. Use this file in both backend (route guards) and frontend (UI visibility)

---

3. Technology Stack

Standard Says

Prefer established, proven technologies. Recommended: Next.js + Tailwind (frontend), Python or Node.js (backend), PostgreSQL (database), Redis (caching), Terraform (IaC), Azure (cloud).

In Practice

The standard's recommended stack is a solid default. Here's the decision tree:

```
What are you building?
|
+-- Internal business tool (CRUD + dashboards)
|   -> Next.js + Tailwind + PostgreSQL
|   -> Backend: Next.js API routes (simplest) or separate NestJS/FastAPI
|
+-- AI/ML application
|   -> Next.js frontend + FastAPI backend (Python)
|   -> Why: Python ecosystem for AI (LangChain, OpenAI SDK, pandas)
|
+-- Data pipeline / automation
```

```
|   -> FastAPI or Azure Functions (Python)
|   -> Frontend only if needed
|
+-- High-traffic API service
    -> NestJS or FastAPI
    -> Consider Azure Container Apps for scaling
```

The principle matters more than the specific tool: Pick something proven with a large community. Don't chase new frameworks.

When to Deviate

- **Team expertise** -- If your team knows Django well, use Django. A framework your team knows beats one that's "better" on paper.
- **Existing codebase** -- Don't rewrite to match the standard. Align new services going forward.

Getting Started

For your first app, pick one column and stick with it:

Layer	Option A (Node.js)	Option B (Python)
Frontend	Next.js + Tailwind	Next.js + Tailwind
Backend	NestJS	FastAPI
Database	PostgreSQL	PostgreSQL
ORM/Driver	Prisma or pg	SQLAlchemy or asyncpg
Auth	openid-client	authlib
Validation	Zod	Pydantic
Sessions	express-session + Redis	Redis-backed sessions

---

4. Multi-Tenancy

Standard Says

Use shared-schema multi-tenancy with `tenant_id` column and Row-Level Security (RLS). Every table has `tenant_id` foreign key.

In Practice

Multi-tenancy is the standard with the most flexibility. **Most internal business tools don't need it.**

You need multi-tenancy when:

- Building a B2B SaaS product
- Multiple customers/organizations share the same deployment
- Data isolation between groups is a requirement

You can skip multi-tenancy when:

- Single organization / internal tool

- Each deployment serves one customer
- You're building a prototype that may become multi-tenant later

When to Deviate

This is the most common valid exception. If you're building an internal tool for your own team, skip it. But **design your schema so** `tenant_id` can be added later -- use proper foreign keys, avoid global assumptions.

Getting Started (if you need it)

1. Add `tenant_id UUID` column to every data table
2. Enable PostgreSQL RLS: `ALTER TABLE x ENABLE ROW LEVEL SECURITY`
3. Create RLS policy: `USING (tenant_id = current_setting('app.current_tenant_id')::uuid)`
4. Set tenant context on every request from the authenticated session

---

5. Security Essentials

Standard Says

Security-first design: Zero Trust networking, Secret Management such as Azure Key Vault for all secrets, input validation on all endpoints, security headers, audit logging, prompt injection protection.

In Practice

Security is non-negotiable, but implementation can be phased. Here's the priority order:

Tier 1: Do These From Day One

Input validation -- Validate all user input with schemas before processing.

```python
# Pydantic (FastAPI) -- you get this for free
class CreateUserRequest(BaseModel):
    email: EmailStr
    display_name: str = Field(min_length=2, max_length=100)
    roles: list[AppRole]
```

```typescript
// Zod (Node.js)
const CreateUserSchema = z.object({
  email: z.string().email(),
  displayName: z.string().min(2).max(100),
  roles: z.array(z.nativeEnum(AppRole)).nonempty(),
});
```

Parameterized queries -- Never concatenate user input into SQL.

```python
# GOOD
await conn.fetch("SELECT * FROM users WHERE email = $1", email)

# BAD -- SQL injection vulnerability
await conn.fetch(f"SELECT * FROM users WHERE email = '{email}'")
```

HTTPS in production -- Always. No exceptions.

Tier 2: Add Before Production

Security headers -- Configure on your web server or in Next.js config:

- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`
- `Strict-Transport-Security: max-age=31536000`
- `Content-Security-Policy: default-src 'self'`

Audit logging -- Log security-relevant events:

```sql
CREATE TABLE IF NOT EXISTS audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id VARCHAR(255),
  user_email VARCHAR(255),
  action VARCHAR(100) NOT NULL,        -- 'login', 'logout', 'create_user', 'delete_key'
  resource_type VARCHAR(100),          -- 'user', 'api_key', 'settings'
  resource_id VARCHAR(255),
  ip_address VARCHAR(45),
  user_agent TEXT,
  details JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_audit_logs_user ON audit_logs(user_email);
CREATE INDEX idx_audit_logs_action ON audit_logs(action);
CREATE INDEX idx_audit_logs_time ON audit_logs(created_at);
```

Log at minimum: login/logout, failed auth attempts, user management changes, settings changes, API key operations.

Secrets management -- In production, use Azure Key Vault (not .env files). In development, .env files are acceptable.

Tier 3: Add For Production Hardening

- Rate limiting on API endpoints
- CORS policy configuration
- Penetration testing (external firm)
- AI prompt injection protection (if using LLMs)
- Zero Trust networking (private endpoints, VNet integration)

When to Deviate

- **Tier 1 items:** Never skip these. They prevent the most common vulnerabilities.
- **Tier 2 items:** Can be deferred for internal prototypes, but must be in place before any external users.
- **Tier 3 items:** Scale-dependent. Add as your user base and threat profile grow.

---

6. Infrastructure as Code

Standard Says

All Azure infrastructure MUST be defined in Terraform (HCL). Use modular structure, remote state backend, never hardcode secrets.

In Practice

Terraform is the right choice for production infrastructure. But for your first app, prioritize getting the application working first, then codify the infrastructure.

Phase your IaC adoption:

1. **First deploy:** Manual Azure portal setup is OK for learning. Document what you clicked.
2. **Second deploy:** Write Terraform to reproduce what you built manually.
3. **Ongoing:** All changes through Terraform. No more portal clicks.

Getting Started

Minimum Terraform structure:

```
infrastructure/
```

```
  main.tf              # Root module
  variables.tf         # Input variables
  outputs.tf           # Output values
  versions.tf          # Provider versions
  backend.tf           # Remote state config
  environments/
    dev.tfvars
    prod.tfvars
```

Non-negotiable Terraform rules:

- Never store secrets in `.tf` files or `.tfvars`
- Use remote state backend (Azure Storage Account)
- Review `terraform plan` before every `terraform apply`

When to Deviate

- **Prototyping phase:** Manual setup is fine. But write it down so you can Terraform it later.
- **Team preference:** Pulumi (TypeScript/Python) is an accepted alternative if the team prefers it. Requires ARB approval per the standards.

---

7. Containerization

Standard Says

Containerize when you have multiple runtimes, need environment consistency, or deploy to Docker or Kubernetes. *Prefer App Service / Functions when containers aren't needed.*

In Practice

Decision tree:

```
Is your app a single runtime (just Node.js OR just Python)?
|
+-- Yes -> Azure App Service (no container needed)
|
+-- No (e.g., Python backend + Node.js frontend + mock-oidc)
    -> Docker Compose for local dev
    -> Azure Container Apps for production
```

Docker Compose for local development is almost always valuable even if you deploy without containers. It gives you reproducible dev environments and one-command startup.

Getting Started

Standard Dockerfile pattern (follow these even if you're new to Docker):

1. **Multi-stage builds** -- separate build from runtime (smaller images)
2. **Non-root user** -- never run as root in the container
3. **Alpine base images** -- 100MB vs 1GB
4. **Copy package files first** -- enables layer caching for faster rebuilds

---

8. M.A.C.H. Architecture

Standard Says

Applications should be Microservices-based, API-first, Cloud-native, and Headless.

In Practice

For most first-time business apps, start with a well-structured monolith. M.A.C.H. principles still apply at a smaller scale:

- **API-first (A):** Define your API contract before building UI. Frontend calls backend API -- no business logic in React components.
- **Cloud-native (C):** Use managed services (PostgreSQL Flexible Server, Redis Cache) instead of self-hosting.
- **Headless (H):** Backend returns JSON, not HTML. Frontend is a separate concern.
- **Microservices (M):** Start as a monolith with clear module boundaries. Extract to microservices only when you need independent scaling or separate team ownership.

When to Deviate

The standards document itself acknowledges M.A.C.H. can be skipped for:

- Solo developer, small app, short lifecycle
- Internal tool with no growth plans
- True prototype (never going to production)

But always maintain API-first and Headless patterns -- these cost nothing extra and make future changes easy.

Quick-Start Checklist

For your first business application, ensure you have:

Before Writing Code

- [ ] Identify your OIDC provider (Azure AD for production, mock-oidc for dev)
- [ ] Choose your stack (see Section 3 decision tree)
- [ ] Define your roles and permissions (see Section 2)
- [ ] Set up a Git repository with `.gitignore` (no secrets committed)

During Development

- [ ] OIDC authentication working (login, callback, me, logout)
- [ ] Permission-based access control with centralized config
- [ ] Input validation on all API endpoints (Pydantic / Zod)
- [ ] Parameterized database queries (no string concatenation)
- [ ] Docker Compose for local development
- [ ] `.env` for local secrets (never committed to Git)

Before Production

- [ ] Security headers configured
- [ ] Audit logging for auth events and admin actions
- [ ] Secrets in Azure Key Vault (not .env)
- [ ] Terraform for infrastructure
- [ ] HTTPS enforced
- [ ] Session timeout enforcement (inactivity + absolute)

Production Hardening

- [ ] Rate limiting on public endpoints

- [ ] CORS policy configured
- [ ] Penetration test (external firm)
- [ ] Monitoring and alerting (Application Insights)
- [ ] Runbook documentation

Standards Compliance Matrix

Use this to track your app's alignment with the Enterprise Standards:

Standard	Required	Flexible	Skip If
OIDC / Azure AD SSO	Yes -- always	Library choice	Never skip
Permission-Based RBAC	Yes -- strongly recommended	Number of roles, granularity	Single-role apps only
Approved Tech Stack	Yes -- for new projects	Specific framework within category	Existing codebase
Input Validation	Yes -- always	Schema library choice	Never skip
Audit Logging	Yes -- before production	Log storage location	True prototype
Security Headers	Yes -- before production	CSP strictness	Local-only tools
Secrets in Key Vault	Yes -- production	Dev can use .env	Local dev only
Terraform / IaC	Yes -- production	Pulumi as alternative	Prototype phase
Multi-Tenancy	Only if B2B SaaS	Isolation strategy	Internal single-org tools
Microservices (M.A.C.H.)	Principle yes, decomposition flexible	Start monolith, extract later	Small team, single app
Containerization	When multi-runtime	Container strategy	Single-runtime on App Service
Zero Trust Networking	Production	Phased implementation	Dev environment

---

Document History

Version	Date	Author	Changes
1.0	2026-03-06	Architecture Team	Initial release bridging Enterprise Standards with practical implementation